Implementación del modelo físico en PostgreSQL

Red Social Estudiantil Pascualina |

Jesus David Vargas Henao

Jose Daniel Vargas Henao

Bases de Datos I

Oralia cortes Grajales

17 de octubre de 2026

1. Objetivo y contexto

La base de datos soporta perfiles de estudiantes, conexiones entre compañeros, habilidades e intereses, publicaciones, grupos, mentorías y eventos. El objetivo es transformar el modelo lógico proporcionado en tablas y restricciones SQL que puedan implementarse en PostgreSQL.

2. Elección del SGBD

Se propone PostgreSQL por ser un gestor relacional de código abierto con soporte robusto de claves foráneas, restricciones CHECK, transacciones y herramientas de administración como pgAdmin. Frente a MySQL, PostgreSQL ofrece un conjunto sólido de funciones SQL avanzadas, aunque el equipo puede necesitar mayor familiarización con su administración. Frente a SQLite, es más adecuado para un escenario multiusuario con servidor, a cambio de requerir instalación y operación del servicio. La elección se basa en los requisitos académicos de integridad y en la posible evolución del proyecto; no supone una prueba de rendimiento.

3. Transformación del modelo lógico al físico

Se conservaron las 15 entidades y sus nombres de campos visibles en el diagrama. Los tipos INT se implementaron como INTEGER, DATETIME como TIMESTAMP, VARCHAR con límites propuestos según su finalidad y TEXT para contenido extenso. Se añadieron restricciones explícitas PRIMARY KEY, FOREIGN KEY, UNIQUE y CHECK cuando corresponde. Las tablas puente utilizan claves primarias compuestas para impedir asociaciones repetidas. Las acciones referenciales son decisiones de diseño propuestas y deben revisarse según la política de conservación de datos del proyecto.

4. Inventario completo de tablas

ESTUDIANTE, PERFIL, HABILIDAD, INTERES, SEGUIR, MENTORIA, GRUPO, PUBLICACION, MIEMBRO_GRUPO, EVENTO, COMENTARIO, REACCION, ASISTENTE_EVENTO, ESTUDIANTE_HABILIDAD, ESTUDIANTE_INTERES. Total: 15 tablas.

5. Descripción de las tablas implementadas

ESTUDIANTE

Campo	Tipo / tamaño	Restricción principal
id_estudiante	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
fecha_registro	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP
ultima_actividad	TIMESTAMP	—

PK: id_estudiante. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

PERFIL

Campo	Tipo / tamaño	Restricción principal
id_perfil	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY

id_estudiante	INTEGER	NOT NULL UNIQUE
biografia	TEXT	—
semestre	INTEGER	—
desorg_perfil	TEXT	—
fecha_actualizacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

PK: id_perfil. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

HABILIDAD

Campo	Tipo / tamaño	Restricción principal
id_habilidad	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
descripcion	TEXT	—

PK: id_habilidad. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

INTERES

Campo	Tipo / tamaño	Restricción principal
id_interes	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
descripcion	TEXT	—

PK: id_interes. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

SEGUIR

Campo	Tipo / tamaño	Restricción principal
id_seguidor	INTEGER	NOT NULL
id_seguido	INTEGER	NOT NULL
fecha_seguimiento	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_seguidor,id_seguido. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el

alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

MENTORIA

Campo	Tipo / tamaño	Restricción principal
id_mentor	INTEGER	NOT NULL
id_aprendiz	INTEGER	NOT NULL
fecha_seguimiento	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_mentor,id_aprendiz. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

GRUPO

Campo	Tipo / tamaño	Restricción principal
id_grupo	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
creador_id	INTEGER	NOT NULL
descripcion	TEXT	—
fecha_creacion	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_grupo. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

PUBLICACION

Campo	Tipo / tamaño	Restricción principal
id_publicacion	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
id_estudiante	INTEGER	NOT NULL
contenido	TEXT	NOT NULL
fecha_publicacion	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_publicacion. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

MIEMBRO_GRUPO

Campo	Tipo / tamaño	Restricción principal

id_estudiante	INTEGER	NOT NULL
id_grupo	INTEGER	NOT NULL
fecha_ingreso	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_estudiante,id_grupo. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

EVENTO

Campo	Tipo / tamaño	Restricción principal
id_evento	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
id_grupo	INTEGER	NOT NULL
descripcion	TEXT	—
fecha_inicio	TIMESTAMP	NOT NULL
fecha_fin	TIMESTAMP	—
fecha_creacion	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_evento. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

COMENTARIO

Campo	Tipo / tamaño	Restricción principal
id_comentario	INTEGER	GENERATED ALWAYS AS IDENTITY PRIMARY KEY
id_publicacion	INTEGER	NOT NULL
id_estudiante	INTEGER	NOT NULL
comentario	TEXT	NOT NULL
fecha_comentario	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_comentario. Se utiliza un identificador estable para distinguir cada registro sin depender de atributos que pueden cambiar.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

REACCION

Campo	Tipo / tamaño	Restricción principal
id_estudiante	INTEGER	NOT NULL

id_publicacion	INTEGER	NOT NULL
fecha	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_estudiante,id_publicacion. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

ASISTENTE_EVENTO

Campo	Tipo / tamaño	Restricción principal
id_estudiante	INTEGER	NOT NULL
id_evento	INTEGER	NOT NULL
fecha_registro	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_estudiante,id_evento. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

ESTUDIANTE_HABILIDAD

Campo	Tipo / tamaño	Restricción principal
id_estudiante	INTEGER	NOT NULL
id_habilidad	INTEGER	NOT NULL
fecha_agregado	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_estudiante,id_habilidad. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

ESTUDIANTE_INTERES

Campo	Tipo / tamaño	Restricción principal
id_estudiante	INTEGER	NOT NULL
id_interes	INTEGER	NOT NULL
fecha_agregado	TIMESTAMP	NOT NULL DEFAULT CURRENT_TIMESTAMP

PK: id_estudiante,id_interes. La clave compuesta identifica una asociación entre entidades y evita duplicados de la misma pareja.

Tipos: INTEGER permite identificadores de hasta aproximadamente 2.147 millones de valores positivos, suficientes para el alcance institucional previsto. VARCHAR(n) expresa un límite funcional en datos breves; TEXT se utiliza en descripciones o contenido variable. TIMESTAMP conserva fecha y hora. Los límites son supuestos de implementación que deben validarse.

6. Relaciones y acciones referenciales

Padre	Hija	FK → PK	Borrado / actualización
estudiante	perfil	id_estudiante → id_estudiante	CASCADE / CASCADE
estudiante	seguir	id_seguidor → id_estudiante	CASCADE / CASCADE
estudiante	seguir	id_seguido → id_estudiante	CASCADE / CASCADE
estudiante	mentoria	id_mentor → id_estudiante	CASCADE / CASCADE
estudiante	mentoria	id_aprendiz → id_estudiante	CASCADE / CASCADE
estudiante	grupo	creador_id → id_estudiante	RESTRICT / CASCADE
estudiante	publicacion	id_estudiante → id_estudiante	CASCADE / CASCADE
estudiante	miembro_grupo	id_estudiante → id_estudiante	CASCADE / CASCADE
grupo	miembro_grupo	id_grupo → id_grupo	CASCADE / CASCADE
grupo	evento	id_grupo → id_grupo	CASCADE / CASCADE
publicacion	comentario	id_publicacion → id_publicacion	CASCADE / CASCADE
estudiante	comentario	id_estudiante → id_estudiante	CASCADE / CASCADE
estudiante	reaccion	id_estudiante → id_estudiante	CASCADE / CASCADE
publicacion	reaccion	id_publicacion → id_publicacion	CASCADE / CASCADE

Padre	Hija	FK → PK	Borrado / actualización
estudiante	asistente_evento	id_estudiante → id_estudiante	CASCADE / CASCADE
evento	asistente_evento	id_evento → id_evento	CASCADE / CASCADE
estudiante	estudiante_habilidad	id_estudiante → id_estudiante	CASCADE / CASCADE
habilidad	estudiante_habilidad	id_habilidad → id_habilidad	CASCADE / CASCADE
estudiante	estudiante_interes	id_estudiante → id_estudiante	CASCADE / CASCADE
interes	estudiante_interes	id_interes → id_interes	CASCADE / CASCADE

Las relaciones se implementan como 1:N mediante claves foráneas en la tabla hija. Las relaciones conceptuales N:M se resuelven mediante tablas intermedias, como estudiante_habilidad y estudiante_interes. La relación ESTUDIANTE–PERFIL se restringe a 1:1 mediante UNIQUE(id_estudiante) en PERFIL.

Justificación de ON DELETE: CASCADE se utiliza cuando se decide eliminar registros dependientes al borrar su entidad padre (por ejemplo, comentarios de una publicación). RESTRICT se utiliza para impedir eliminar un estudiante que siga siendo creador de un grupo. Estas políticas son propuestas; un sistema real podría requerir desactivación lógica, retención de auditoría o reasignación de propiedad.

Justificación de ON UPDATE: CASCADE mantiene las referencias coherentes si cambia una clave primaria, aunque normalmente los identificadores internos no se modifican.

7. Estructura del script DDL

El archivo modelo_fisico_pascualina.sql contiene una transacción BEGIN/COMMIT y 15 sentencias CREATE TABLE ordenadas para crear primero las tablas padre y después las dependientes. Cada tabla define sus campos, tipos, claves y reglas de integridad. Se recomienda ejecutarlo una sola vez en una base vacía y guardar evidencia del resultado.

```
CREATE TABLE estudiante (
 id_estudiante INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 nombre VARCHAR(100) NOT NULL,
 apellido VARCHAR(100) NOT NULL,
 correo VARCHAR(254) NOT NULL UNIQUE,
 password_hash VARCHAR(255) NOT NULL,
 fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
 ultima_actividad TIMESTAMP,
 estado VARCHAR(20) NOT NULL DEFAULT 'activo',
 CONSTRAINT ck_estudiante_estado CHECK (estado IN ('activo','inactivo','suspendido'))
);
-- Perfil: uno por estudiante (UNIQUE en la FK implementa 1:1).
CREATE TABLE perfil (
 id_perfil INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 id_estudiante INTEGER NOT NULL UNIQUE,
 area_estudio VARCHAR(120), biografia TEXT, programa VARCHAR(120),
 semestre INTEGER, ciudad VARCHAR(100), desorg_perfil TEXT,
 fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT ck_perfil_semestre CHECK (semestre IS NULL OR semestre >= 1),
 CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante)
 ON UPDATE CASCADE ON DELETE CASCADE
);
-- Catalogo de habilidades.
CREATE TABLE habilidad (
 id_habilidad INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 nombre VARCHAR(120) NOT NULL, descripcion TEXT
);
-- Catalogo de intereses.
CREATE TABLE interes (
 id_interes INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 nombre VARCHAR(120) NOT NULL, descripcion TEXT
);
-- Seguimiento entre estudiantes: PK compuesta impide duplicar la pareja.
```

```
CREATE TABLE seguir (
 id_seguidor INTEGER NOT NULL, id_seguido INTEGER NOT NULL,
 fecha_seguimiento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT pk_seguir PRIMARY KEY (id_seguidor,id_seguido),
 CONSTRAINT ck_seguir_distintos CHECK (id_seguidor <> id_seguido),
 CONSTRAINT fk_seguir_seguidor FOREIGN KEY (id_seguidor) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,
 CONSTRAINT fk_seguir_seguido FOREIGN KEY (id_seguido) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE
);
-- Mentorias: PK compuesta de mentor y aprendiz.
CREATE TABLE mentoria (
 id_mentor INTEGER NOT NULL, id_aprendiz INTEGER NOT NULL,
 fecha_seguimiento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT pk_mentoria PRIMARY KEY (id_mentor,id_aprendiz),
 CONSTRAINT ck_mentoria_distintos CHECK (id_mentor <> id_aprendiz),
 CONSTRAINT fk_mentoria_mentor FOREIGN KEY (id_mentor) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,
 CONSTRAINT fk_mentoria_aprendiz FOREIGN KEY (id_aprendiz) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE
);
-- Grupo: el creador debe ser un estudiante existente.
CREATE TABLE grupo (
 id_grupo INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 creador_id INTEGER NOT NULL, nombre VARCHAR(120) NOT NULL,
 descripcion TEXT, tipo VARCHAR(50),
 fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
 estado VARCHAR(20) NOT NULL DEFAULT 'activo',
 CONSTRAINT ck_grupo_estado CHECK (estado IN ('activo','inactivo','cerrado')),
 CONSTRAINT fk_grupo_creador FOREIGN KEY (creador_id) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE RESTRICT
);
-- Publicacion: autor existente.
CREATE TABLE publicacion (
 id_publicacion INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 id_estudiante INTEGER NOT NULL, contenido TEXT NOT NULL,
 tipo VARCHAR(50), fecha_publicacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
 CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Interseccion estudiante-grupo, con PK compuesta.

CREATE TABLE miembro_grupo (

 id_estudiante INTEGER NOT NULL, id_grupo INTEGER NOT NULL,

 fecha_ingreso TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, rol VARCHAR(50),

 CONSTRAINT pk_miembro_grupo PRIMARY KEY (id_estudiante,id_grupo),

 CONSTRAINT fk_miembro_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,

 CONSTRAINT fk_miembro_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Evento: cada evento pertenece a un grupo.

CREATE TABLE evento (

 id_evento INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

 id_grupo INTEGER NOT NULL, nombre VARCHAR(120) NOT NULL,

 descripcion TEXT, fecha_inicio TIMESTAMP NOT NULL, fecha_fin TIMESTAMP,

 lugar VARCHAR(150), modalidad VARCHAR(50), organizador VARCHAR(120),

 fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 CONSTRAINT ck_evento_fechas CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio),

 CONSTRAINT fk_evento_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Comentario: depende de publicacion y estudiante.

CREATE TABLE comentario (

 id_comentario INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

 id_publicacion INTEGER NOT NULL, id_estudiante INTEGER NOT NULL,

 comentario TEXT NOT NULL, fecha_comentario TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 CONSTRAINT fk_comentario_publicacion FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion) ON UPDATE CASCADE ON DELETE CASCADE,

 CONSTRAINT fk_comentario_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Reaccion: una por estudiante y publicacion, segun PK compuesta del diagrama.

CREATE TABLE reaccion (

 id_estudiante INTEGER NOT NULL, id_publicacion INTEGER NOT NULL,
```

```
 tipo_reaccion VARCHAR(50) NOT NULL, fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 CONSTRAINT pk_reaccion PRIMARY KEY (id_estudiante,id_publicacion),

 CONSTRAINT fk_reaccion_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,

 CONSTRAINT fk_reaccion_publicacion FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Asistencia a eventos: PK compuesta evita inscripciones repetidas.

CREATE TABLE asistente_evento (

 id_estudiante INTEGER NOT NULL, id_evento INTEGER NOT NULL,

 estado VARCHAR(50), fecha_registro TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 CONSTRAINT pk_asistente_evento PRIMARY KEY (id_estudiante,id_evento),

 CONSTRAINT fk_asistente_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,

 CONSTRAINT fk_asistente_evento FOREIGN KEY (id_evento) REFERENCES evento(id_evento) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Habilidades por estudiante: tabla puente N:M.

CREATE TABLE estudiante_habilidad (

 id_estudiante INTEGER NOT NULL, id_habilidad INTEGER NOT NULL,

 nivel VARCHAR(50), fecha_agregado TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 CONSTRAINT pk_estudiante_habilidad PRIMARY KEY (id_estudiante,id_habilidad),

 CONSTRAINT fk_eh_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,

 CONSTRAINT fk_eh_habilidad FOREIGN KEY (id_habilidad) REFERENCES habilidad(id_habilidad) ON UPDATE CASCADE ON DELETE CASCADE

);

-- Intereses por estudiante: tabla puente N:M.

CREATE TABLE estudiante_interes (

 id_estudiante INTEGER NOT NULL, id_interes INTEGER NOT NULL,

 fecha_agregado TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

 CONSTRAINT pk_estudiante_interes PRIMARY KEY (id_estudiante,id_interes),

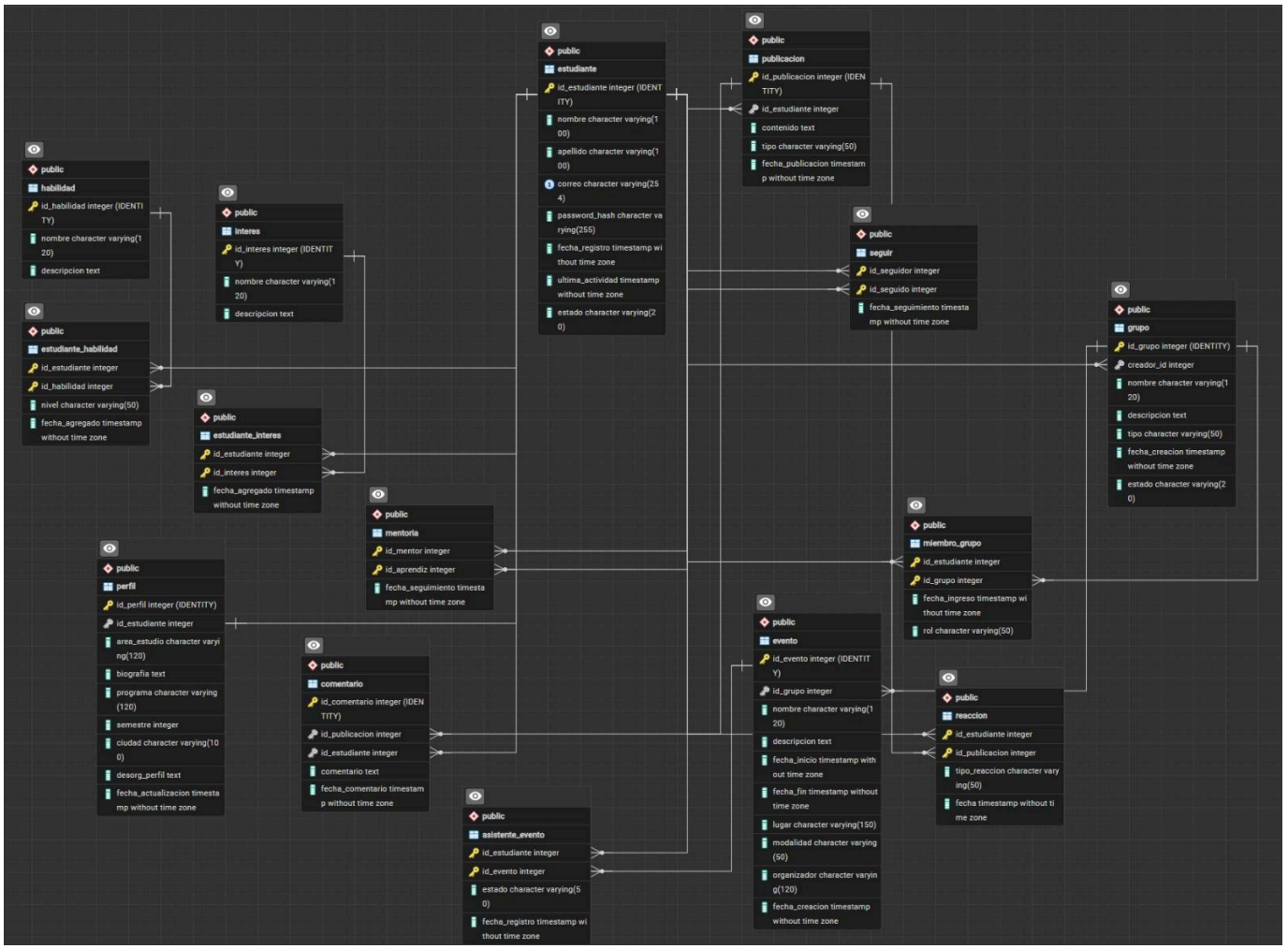
 CONSTRAINT fk_ei_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON UPDATE CASCADE ON DELETE CASCADE,

 CONSTRAINT fk_ei_interes FOREIGN KEY (id_interes) REFERENCES interes(id_interes) ON UPDATE CASCADE ON DELETE CASCADE

);

COMMIT;
```

8. Evidencias de implementación



9. Conclusiones individuales

Jesus: Pasar del diagrama a PostgreSQL me hizo notar que no basta con conectar tablas: hay que decidir qué campos son obligatorios, cómo se identifican los registros y qué ocurre al borrar información. La parte que más tuve que revisar fue el orden de creación y las claves foráneas, porque una tabla no puede depender de otra que todavía no existe.

Jose: En esta actividad entendí mejor por qué los tipos de datos y las restricciones afectan la calidad de una base de datos. Me pareció importante comprobar las relaciones de las tablas puente, porque allí se puede repetir información si no se define bien la clave primaria. También aprendí que una decisión como ON DELETE CASCADE no se debe poner sin pensar en sus consecuencias.